



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Ingegneria del Software

SunSpot: Gestionale per la prenotazione digitale di stabilimenti
balneari

Soldani Leonardo - 7139732
Neamtu Narcis Daniel - 7133414

Indice

1	Introduzione	1
2	Ambiente e tecnologie utilizzate	1
2.1	Linguaggio e Tool di Build	1
2.2	Database e Persistenza	1
2.3	Librerie e Containerizzazione	1
2.4	Testing e Analisi	1
2.5	Strumenti di creazione diagrammi	1
2.6	Intelligenza artificiale	2
3	Use Cases	3
3.1	Utenti del sistema	3
3.2	Descrizione Use Cases	4
3.2.1	Manage User Account	5
3.2.2	Create Booking	6
3.2.3	Manage Reviews	7
3.2.4	Create Beach	8
3.2.5	Manage Beach	9
4	Progettazione	10
4.1	Pattern Architeturali e Progettuali	10
4.1.1	Architettura Esagonale (Ports & Adapters)	10
4.1.2	DDD-lite (Domain-Driven Design)	11
4.1.3	Builder Pattern	13
4.2	Struttura del codice	14
4.3	Domain	15
4.4	Application	16
4.4.1	Port-Out	16
4.4.2	Port-In	17
5	Database	18
5.1	Modello ER	18
5.2	Dizionario dei dati	19
5.3	Tabelle delle Regole	21
6	Mockup	22
7	Unit Testing	26
7.1	Strategia e Obiettivi del Testing	26
7.2	Copertura dei Test: Classi e Metodi	26
7.3	Esecuzione dei Test tramite JUnit 5 Suites	28

1 Introduzione

Il progetto SunSpot nasce dall'esigenza di **digitalizzare e modernizzare** il settore degli stabilimenti balneari, un ambito che ancora oggi mostra una certa resistenza all'adozione di piattaforme web integrate.

L'obiettivo principale è fornire un'infrastruttura capace di permettere all'utente di **prenotare in modo rapido e intuitivo** un ombrellone presso uno dei lidi aderenti al servizio, trasformando la pianificazione di una giornata al mare in un'operazione a portata di click, offrendo al contempo ai gestori uno strumento completo per il controllo della propria attività.

Per garantire un **ecosistema sicuro e affidabile**, SunSpot integra un sistema di regolazione basato su recensioni verificate e un meccanismo di segnalazione (report) tra utenti e gestori, volto a preservare l'integrità del servizio e a **migliorare costantemente l'esperienza d'uso** per tutti gli attori coinvolti.

2 Ambiente e tecnologie utilizzate

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie:

2.1 Linguaggio e Tool di Build

- **Java 25** - Scelto per l'utilizzo delle funzionalità più recenti della JDK.
- **Maven 3.8.5** - Strumento di build e gestione delle dipendenze.

2.2 Database e Persistenza

- **MySQL 8.3.0** - Database relazionale utilizzato per la persistenza dei dati.
- **MySQL Connector/J** - Driver ufficiale per la gestione della connessione al database.

2.3 Librerie e Containerizzazione

- **BCrypt 0.10.2** - Utilizzata per l'hashing e la gestione sicura delle password.
- **Docker & Docker Compose 3.8** - Utilizzati per containerizzare il database MySQL, garantendo un ambiente di esecuzione isolato, persistente e facilmente riproducibile.

2.4 Testing e Analisi

- **JUnit 5.12.1** - Framework per l'esecuzione dei test unitari.
- **Mockito 5.23.0** - Utilizzato per la simulazione di dipendenze durante i test (mocking).
- **JUnit Platform Suite** - Strumento per l'organizzazione e l'esecuzione dei test.

2.5 Strumenti di creazione diagrammi

Per la realizzazione dei diagrammi abbiamo utilizzato i seguenti strumenti:

- **StarUML**: utilizzato per la realizzazione dei diagrammi UML delle classi.
- **draw.io**: impiegato per la creazione dei diagrammi degli Use Case e il diagramma del Modello Entity-Relationship del database.

2.6 Intelligenza artificiale

A supporto dello sviluppo del progetto sono stati utilizzati i modelli linguistici *Gemini 3* e *Gemini 3.1*. L'impiego di tali strumenti si è concentrato sui seguenti ambiti:

- **Architettura progetto:** dopo una prima stesura del diagramma UML, sono stati consultati i modelli di intelligenza artificiale citati al fine di cercare di determinare un'architettura implementativa adatta al progetto. Questo ha, alla fine, portato alla scelta e all'adozione dell'architettura Esagonale. Durante lo sviluppo del progetto, il consulto con tali modelli ha portato anche al suggerimento dell'introduzione di ulteriori pattern per la realizzazione del progetto come il CQRS. (*Command Query Responsibility Segregation*) per ottimizzare e separare le operazioni di lettura da quelle di scrittura nel lato interfacce/database.
- **Generazione codice ripetitivo e strutturale:** al fine di ottimizzare i tempi di sviluppo, l'intelligenza artificiale è stata utilizzata per la generazione di codice strutturale e ripetitivo (es. pattern *Builder*). Durante la progettazione delle modalità di orchestrazione tra il lato Java e il lato SQL, Gemini ha suggerito l'utilizzo di un *TransactionManager* al fine di evitare dipendenze critiche da specifiche librerie e astraendo il processo di gestione delle transazioni SQL indipendentemente dalle librerie o dal database utilizzato con una semplice chiamata ad una funzione Lambda. L'output generato è stato costantemente sottoposto a revisione umana al fine di garantirne correttezza, integrità e coerenza architetturale.
- **Testing:** i modelli sono stati impiegati come supporto per la stesura dei casi di test e per una verifica finale sulla copertura e completezza della suite di testing, fornendo linee guida su cosa testare e quali classi includere nei test. Ciò ha consentito di ottenere una copertura metodica dei percorsi sia di successo (*Happy Paths*) che di fallimento (*Sad Paths*).
- **Prototipazione UI:** per la generazione dei mockup dell'interfaccia, l'IA ha analizzato i limiti dei generatori di immagini tradizionali (che tendono ad avere un resa grafica non soddisfacente) suggerendo l'adozione di piattaforme generative specializzate nella produzione di codice frontend; nello specifico *v0.dev* e *bolt.new*. Il modello linguistico ha inoltre aiutato nella stesura dei prompt da passare alle piattaforme assicurandone la coerenza con gli Use Case e le Business Rules del progetto.

3 Use Cases

3.1 Utenti del sistema

Il programma coinvolge tre attori principali: **Customer**, **Owner** e **Admin**. Ciascuno di essi ha la possibilità di effettuare azioni specifiche per il proprio ruolo:

- **Customer**: utente finale dell'applicazione. Ha la possibilità di **effettuare prenotazioni** scegliendo l'ombrellone desiderato; può **richiedere** inoltre **servizi extra** come lettini, sdraio e sedie, o la prenotazione di un camerino. Questa tipologia di utente può **rilasciare** un'unica **recensione** per ogni spiaggia presso cui ha prenotato e può **inviare una segnalazione** (report) agli admin qualora riscontri violazioni delle linee guida da parte di uno stabilimento.
- **Owner**: gestore dello stabilimento balneare. Ha il compito di **configurare** il proprio lido partendo dalla segnalazione dei servizi comuni, passando per **l'organizzazione** geografica della spiaggia **in zone** con diverse fasce di prezzo, fino alla possibilità di **definire stagionalità** con diverse tariffe. Può inoltre **effettuare report** nei confronti dei customer che hanno tenuto comportamenti scorretti, come ad esempio il mancato rispetto di una prenotazione.
- **Admin**: svolge il ruolo di gestore dell'ambiente commerciale offerto dall'applicazione, **accogliendo** i report e **valutandoli** tramite l'accesso alla cronologia dei report inviati e ricevuti dagli utenti, con la possibilità di accettarli o rifiutarli. Al fine di gestire l'ambiente, l'admin utilizza lo strumento del **ban**, che può colpire sia i customer che gli owner/spiagge. Il ban può essere di due tipi: da una **singola spiaggia** (applicabile solo ai customer) oppure **dall'intera applicazione** (applicabile a entrambi).

3.2 Descrizione Use Cases

Il programma si pone l'obiettivo di implementare un'applicazione a scopi commerciali; di conseguenza l'azione di eliminare prenotazioni, spiagge, utenti ed altre informazioni di natura fiscale risulterebbe un comportamento superficiale da parte dei programmatori. Per questo motivo, il sistema non prevede l'eliminazione fisica di record sensibili, ma utilizza un meccanismo di disattivazione logica.

Di seguito (Figura 1) è riportato il **diagramma degli Use Cases** del progetto, nelle sezioni successive verranno analizzati nel dettaglio i flussi più rilevanti. Ad ogni tabella Use Case, è stata aggiunta una sezione "**Business Rules e Riferimenti**", contenente le regole di Business associate e riferimenti a (eventuali) Mockups e Test correlati.

Per visualizzare tutti gli Use Cases nel dettaglio, si consulti [il seguente PDF](#).

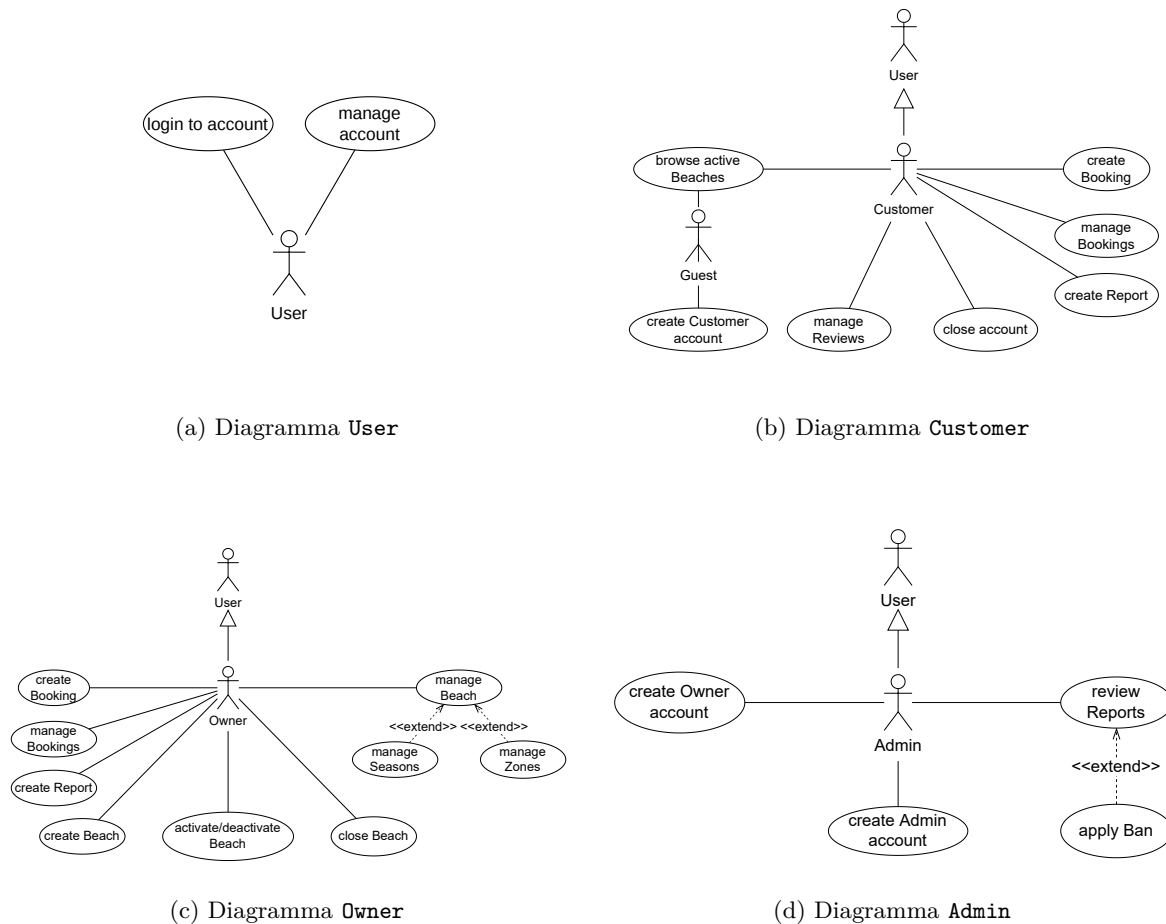


Figura 1: Diagramma degli Use Cases

3.2.1 Manage User Account

ID Use Case	UC-02
Nome Use Case	Manage Account
Attori	User (Admin, Customer, Owner)
Descrizione	Permette all'utente di modificare i suoi dati legati all'account (username, email, numero di telefono, indirizzo ecc.).
Precondizioni	L'utente in questione deve aver fatto login inizialmente all'account.
Postcondizioni	I dati modificati vengono aggiornati nel database e viene mostrato all'utente un messaggio di conferma dei cambiamenti.
Flusso Principale	1. L'utente naviga dentro la pagina "Impostazioni Account/Modifica informazioni"; 2. Il sistema recupera i dati e mostra i dati attuali dell'account (nome, cognome); 3. L'utente modifica i dati che preferisce; 4. L'utente clicca su "Salva modifiche"; 5. Il sistema convalida e salva i nuovi dati; 6. Viene mosrato all'utente un messaggio di conferma.
Flussi alternativi	2a. L'utente è un Customer: Il sistema mostra anche l'indirizzo; 3a. L'utente è un Customer: L'utente può modificare anche l'indirizzo; 1a. Cambio email (da passo 1): i. L'utente naviga su "Cambia Email"; ii. Il sistema mostra una pagina dedicata per il cambio email; iii. L'utente inserisce la nuova email e la password attuale; iv. Il sistema verifica l'unicità della email nel database e aggiorna la nuova email nel database. 1b-e. Email già usata per un altro account/non valida: Il sistema segnala l'errore all'utente e invita a ricontrollare. 1b-e. Password non valida: Il sistema segnala l'errore all'utente e invita a reinserire la password. 1c. Cambio numero di telefono (L'utente è un Customer, da passo 1): i. L'utente naviga su "Cambia numero di telefono"; ii. Il sistema mostra una pagina dedicata per il cambio numero di telefono; iii. L'utente inserisce il nuovo numero di telefono e la password attuale; iv. Il sistema verifica l'unicità del numero nel database e aggiorna il nuovo numero nel database. 1c-e. Numero di telefono già usato per un altro account/non valido: Il sistema segnala l'errore all'utente e invita a ricontrollare. 1c-e. Password non valida: Il sistema segnala l'errore all'utente e invita a reinserire la password. 1d. Cambio password (da passo 1): i. L'utente naviga su "Cambia password"; ii. Il sistema mostra una pagina dedicata per il cambio password; iii. L'utente inserisce la nuova e la vecchia password; iv. Il sistema verifica la password vecchia; v. Il sistema aggiorna la password nel database e mostra un messaggio di conferma all'utente. 1d-e. Password vecchia non valida: Il sistema segnala l'errore all'utente e invita a reinserire la vecchia password.
Business Rules e Riferimenti	• Ogni utente deve inserire dati validi. I dati non possono essere nulli. • Solo Customer possiede indirizzo e numero di telefono; • Username, nome, cognome (per i Customers: indirizzo) possono essere cambiati allo stesso momento. Email, Password (per i Customers: numero di telefono) invece devono essere cambiati singolarmente attraversando dei check di sicurezza. • Testing: le classi che si occupano dei test sono <code>ManageUserServiceTest</code>, <code>ManageCustomerServiceTest</code>, <code>ManageOwnerServiceTest</code> e <code>ManageAdminServiceTest</code> nel package <code>service</code>; <code>UserTest</code>, <code>CustomerTest</code>, <code>OwnerTest</code> e <code>AdminTest</code> nel package <code>domain</code>. Vengono usate anche <code>service.AddressServiceTest</code> e <code>domain.AddressTest</code> in caso di un <code>Customer</code> che decide di modificare l'indirizzo.

Figura 2: UC: Manage Account

3.2.2 Create Booking

ID Use Case	UC-05
Nome Use Case	Create Booking
Attori	Customer, Owner
Descrizione	Permette ad un Customer di prenotare posti, servizi extra e parcheggi presso una spiaggia in una certa data. Permette inoltre ad un Owner di registrare manualmente una prenotazione per un cliente telefonico.
Precondizioni	L'utente deve aver effettuato il login. Se l'utente è un Customer, non deve essere bannato dalla spiaggia selezionata. Se l'utente è un Owner, deve possedere una spiaggia attiva. La spiaggia selezionata (o posseduta dall'Owner) deve essere nello stato ATTIVO.
Postcondizioni	Viene creato un nuovo record di prenotazione nel database con un prezzo totale calcolato e uno stato iniziale (PENDING per i Customer, CONFIRMED per gli Owner).
Flusso Principale	1. Il Customer seleziona una spiaggia (partendo da UC-04) e sceglie una data; 2. Il sistema verifica che il Customer non sia bannato da quella spiaggia; 3. Il sistema mostra la mappa o la lista delle disponibilità per la data scelta; 4. Il Customer seleziona i posti spiaggia desiderati (es. ombrelloni, lettini); 5. Il Customer seleziona eventuali extra (lettini, sdraio, sedie); 6. Il Customer inserisce il numero di parcheggi desiderati; 7. Il sistema calcola e mostra il prezzo totale riepilogativo; 8. Il Customer conferma la prenotazione; 9. Il sistema salva la prenotazione e le assegna lo stato "PENDING", mostrando un messaggio di successo.
Flussi alternativi	1a. Flusso Owner (Prenotazione Telefonica): i. L'Owner accede al pannello gestionale della propria spiaggia e avvia una nuova prenotazione; ii. L'Owner seleziona la data, i posti, gli extra e i parcheggi (come nei passi 3-6 del Main Flow); iii. L'Owner inserisce obbligatoriamente il Nome e il Numero di Telefono del cliente chiamante; iv. Il sistema calcola e mostra il totale; v. L'Owner conferma la prenotazione; vi. Il sistema salva la prenotazione assegnandole automaticamente lo stato "CONFIRMED". 2a-e. Utente Bannato: Il sistema rileva che il Customer è nella blacklist della spiaggia e blocca l'operazione mostrando il messaggio: "Non sei autorizzato a prenotare in questa struttura." 4/6a-e. Disponibilità Esaurita: Se durante la selezione i posti o i parcheggi scelti vengono esauriti (es. prenotati da un altro utente nello stesso istante), il sistema avvisa l'utente e lo invita a modificare la scelta.
Business Rules e Riferimenti	• Le prenotazioni dei Customer partono sempre dallo stato PENDING e richiedono un passaggio a CONFIRMED da parte dell'Owner. • Le prenotazioni create dall'Owner sono considerate già confermate (CONFIRMED). • L'Owner non può selezionare la spiaggia: il sistema forza la prenotazione solo sullo stabilimento di sua proprietà. • Il calcolo del totale si basa sul tariffario (Season/Zone) impostato per quella data. • Mockup: si veda Figura 16 - Flusso di Prenotazione • Testing: le classi che si occupano dei test sono <code>CreateBookingServiceTest</code> e <code>CreateManualBookingServiceTest</code> nel package <code>service</code>; <code>BookingTest</code>, <code>BookingParkingTest</code> e <code>PriceCalculatorTest</code> nel package <code>domain</code>.

Figura 3: UC: Create Booking

3.2.3 Manage Reviews

ID Use Case	UC-07
Nome Use Case	Manage Reviews
Attori	Customer
Descrizione	Permette ad un Customer di valutare e commentare la propria esperienza presso una spiaggia in cui ha effettivamente soggiornato, oppure di eliminare una propria recensione precedentemente pubblicata.
Precondizioni	L'utente è loggato. La spiaggia è in stato ATTIVO. L'utente non ha un Ban per quella spiaggia. Deve esistere almeno una prenotazione per quella spiaggia con data nel passato (rispetto a quando viene applicato lo Use Case) e in stato CONFIRMED.
Postcondizioni	La recensione viene salvata o rimossa dal database, associata alla spiaggia e all'utente.
Flusso Principale	1. L'utente naviga nella sezione "Le mie prenotazioni" o sulla pagina della spiaggia; 2. L'utente clicca sul pulsante "Lascia una recensione" per la spiaggia selezionata; 3. Il sistema verifica che tutti i requisiti siano soddisfatti (prenotazione passata e CONFIRMED, assenza di ban, spiaggia attiva); 4. Il sistema mostra il modulo della recensione; 5. L'utente seleziona una valutazione (da 1 a 5 stelle) e inserisce un commento testuale; 6. L'utente clicca su "Pubblica"; 7. Il sistema salva la recensione nel database e mostra un messaggio di conferma.
Flussi alternativi	2a. Eliminazione Recensione: i. L'utente seleziona una recensione che ha già pubblicato in passato e clicca su "Elimina"; ii. Il sistema chiede conferma dell'operazione ("Sei sicuro di voler eliminare questa recensione?"); iii. L'utente conferma; iv. Il sistema rimuove il record dal database e mostra un messaggio di successo. 3a-e. Mancanza Requisiti: Se l'utente tenta di accedere all'URL della recensione senza averne diritto (es. prenotazione futura, prenotazione CANCELLED/REJECTED o mai stato in quella spiaggia), il sistema blocca l'azione mostrando: "Devi aver completato un soggiorno in questa struttura per poter lasciare una recensione.". 3b-e. Spiaggia Disattivata/Ban: Se la spiaggia non è più attiva o l'utente è stato bannato da quella struttura dopo il soggiorno, il sistema inibisce la funzione di recensione. 6a-e. Recensione non completata: Se l'utente non compila entrambi i campi, il sistema evidenzia i campi da compilare obbligatoriamente.
Business Rules e Riferimenti	• La valutazione è obbligatoria ed è un valore intero tra 1 e 5. • Il commento testuale è anch'esso obbligatorio. • Il sistema accetta recensioni solo per prenotazioni la cui data è strettamente minore della data odierna (es. prenotazione il 19 luglio, recensibile dal 20 luglio). • L'App Ban è già gestito a monte dal Login (UC-01). • Mockup: si veda Figura 17 - Le Mie Prenotazioni e Recensioni • Testing: le classi che si occupano dei test sono <code>ReviewServiceTest</code> nel package <code>service</code>; <code>ReviewTest</code> nel package <code>domain</code>.

Figura 4: UC: Manage Reviews

3.2.4 Create Beach

ID Use Case	UC-10
Nome Use Case	Create Beach
Attori	Owner
Descrizione	Permette a un Owner appena registrato (o senza strutture associate) di inserire nel sistema la propria spiaggia, configurandone dati anagrafici, inventario, servizi, parcheggi, stagioni e zone.
Precondizioni	L'utente deve aver effettuato il login come Owner. Nel database NON deve esistere alcuna spiaggia già associata a questo account.
Postcondizioni	Viene creato un nuovo record "Spiaggia" nel database, associato all'Owner. Lo stato sarà ATTIVO se configurata completamente e confermata, altrimenti INATTIVO (bozza).
Flusso Principale	1. L'Owner accede al sistema; il sistema rileva l'assenza di strutture e mostra il pulsante "Crea Spiaggia"; 2. L'Owner clicca sul pulsante e accede al modulo di creazione; 3. L'Owner inserisce i Dati Obbligatori: Nome, Descrizione, Numero di Telefono, Indirizzo e Informazioni Extra; 4. L'Owner (opzionalmente) compila l'Inventario: quantità di ombrelloni, tende, sdraio, lettini, sedie, camerini; 5. L'Owner (opzionalmente) seleziona i Servizi: bagni, docce, piscina, bar, ristorante, Wi-Fi, campo da pallavolo; 6. L'Owner (opzionalmente) configura il Parcheggio: numero posti auto, moto, bici, veicoli elettrici e presenza telecamere di videosorveglianza; 7. L'Owner (opzionalmente) crea le Stagioni (prezzi base e tariffe sulle varie zone) e le Zone (layout ombrelloni/tende); 8. L'Owner clicca su "Salva Spiaggia"; 9. Il sistema convalida i dati obbligatori; 10. Il sistema verifica la completezza della configurazione (presenza sia dei dati obbligatori che di quelli opzionali/operativi); 11. Avendo inserito tutti i dati, il sistema mostra un prompt: "Vuoi attivare la spiaggia e renderla visibile ora?"; 12. L'Owner accetta (o rifiuta); 13. Il sistema salva definitivamente la spiaggia nel database (in stato ATTIVO se l'Owner ha accettato, altrimenti INATTIVO).
Flussi alternativi	9a-e. Campi Obbligatori Mancanti: Se Nome, Indirizzo o altri campi base sono vuoti, il sistema evidenzia gli errori e impedisce il salvataggio. 10a. Salvataggio Parziale (Bozza): Se l'Owner ha inserito i dati obbligatori ma ha saltato configurazioni chiave (es. Stagioni e Zone), il sistema non mostra il prompt di attivazione. Salva direttamente la spiaggia in stato INATTIVO. L'Owner dovrà usare "Manage Beach" (UC-11) in futuro per completarla e attivarla.
Business Rules e Riferimenti	• Relazione 1:1 rigida: un Owner può possedere e gestire al massimo una spiaggia nel sistema. • Per poter passare allo stato ATTIVO, la spiaggia deve possedere almeno i requisiti minimi operativi (Inventario, Servizi e Parcheggi definiti; almeno una Stagione e una Zona configurate, come definito dalle regole di business generali). • Testing: le classi che si occupano dei test sono <code>CreateBeachServiceTest</code> e <code>AddressServiceTest</code> nel package <code>service</code>, tutte le classi nei packages <code>domain.beach</code> e <code>domain.common</code>.

Figura 5: UC: Create Beach

3.2.5 Manage Beach

ID Use Case	UC-11
Nome Use Case	Manage Beach
Attori	Owner
Descrizione	Permette all'Owner di aggiornare i dati generali, l'inventario, i servizi e i parcheggi, nel rigoroso rispetto dello stato di attività della spiaggia.
Precondizioni	L'Owner ha effettuato il login, possiede una spiaggia associata e l'account/spiaggia non sono soggetti a Ban (già verificato al login).
Postcondizioni	Il database viene aggiornato con le nuove informazioni della spiaggia, dell'inventario, dei servizi e/o dei parcheggi.
Flusso Principale	1. L'Owner accede alla dashboard di Gestione Spiaggia; 2. Il sistema recupera i dati attuali e verifica lo stato della spiaggia (ATTIVO o INATTIVO); 3. Il sistema "blocca" (sola lettura) o abilita i campi di modifica in base allo stato attuale; 4. L'Owner naviga tra le sezioni e applica le modifiche desiderate (Anagrafica, Inventario, Servizi, Parcheggi); 5. L'Owner clicca su "Salva Modifiche"; 6. Il sistema esegue i controlli di integrità (es. validazione dell'inventario contro le esigenze delle stagioni future e capienza parcheggi per le prenotazioni attive); 7. Il sistema salva i dati aggiornati nel database e mostra un messaggio di successo.
Flussi alternativi	3a. Spiaggia in stato ATTIVO (Restrizioni): Il sistema rende in sola lettura: Nome, Descrizione, Numero di Telefono, Indirizzo, Inventario, Servizi e Parcheggio. Non è permessa alcuna alterazione strutturale finché la spiaggia riceve prenotazioni. 3b. Spiaggia in stato INATTIVO (Permessi Estesi): Il sistema sblocca la modifica dei dati base, Inventario, Servizi e Parcheggio. 6a-e. Conflitto Inventario: Se la spiaggia è INATTIVA e l'Owner riduce la quantità di inventario (ombrelloni, tende), il sistema verifica le stagioni attuali e future. Se la nuova capacità è inferiore a quella utilizzata nelle Stagioni, il sistema blocca il salvataggio mostrando: "Inventario insufficiente: capacità ridotta sotto la soglia richiesta dalle stagioni programmate." 6b-e. Conflitto Parcheggi: Se l'Owner riduce il numero di parcheggi, il sistema somma i posti auto richiesti da tutte le prenotazioni PENDING e CONFIRMED per ogni singola data (odierna e futura). Se il nuovo limite inserito è inferiore al totale già prenotato in una qualsiasi di queste date, il sistema blocca l'operazione mostrando: "Capacità parcheggio insufficiente per coprire le prenotazioni già attive o in attesa."
Business Rules e Riferimenti	• Dati anagrafici, Inventario, Servizi e Parcheggio sono modificabili esclusivamente in stato INATTIVO. • I controlli di capienza (Inventario e Parcheggi) devono garantire la retrocompatibilità sia con le Stagioni in corso e quelle in futuro che con le prenotazioni già accettate o in approvazione (CONFIRMED/PENDING). • Testing: le classi che si occupano dei test sono <code>ManageBeachServiceTest</code> nel package <code>service</code>; <code>BeachTest</code>, <code>BeachGeneralTest</code>, <code>BeachServicesTest</code>, <code>BeachInventoryTest</code> e <code>ParkingTest</code> nel package <code>domain</code>.

Figura 6: UC: Manage Beach

4 Progettazione

4.1 Pattern Architetture e Progettuali

Per lo sviluppo di SunSpot sono stati adottati diversi pattern volti a garantire un'architettura robusta, testabile e facilmente manutenibile, separando nettamente le responsabilità tra logica di business e dettagli implementativi.

4.1.1 Architettura Esagonale (Ports & Adapters)

Il progetto SunSpot adotta i **principi dell'Architettura Esagonale** per garantire una separazione delle responsabilità e un totale isolamento della logica di business dalle tecnologie esterne. L'architettura prevede un nucleo centrale (**core**), composto dai package **domain** e **application**, e un perimetro esterno, composto dagli **adapters** nel package **infrastructure.persistence.jdbc**.

Lo strato di **application** funge da ponte tra il mondo esterno e il dominio ed è suddiviso in due componenti fondamentali:

- **Ports (interfacce)**: rappresentano i contratti di comunicazione dell'esagono. Si distinguono in:
 - **Port-In (driving ports)**: definiscono i casi d'uso e le azioni che gli attori esterni possono richiedere al sistema (es. Listing 1).
 - **Port-Out (driven ports)**: definiscono i requisiti infrastrutturali necessari al **core** per funzionare (es. Listing 2).
- **Services (orchestratori)**: rappresentano le implementazioni concrete delle port-in. Il loro ruolo è quello di **orchestratori dei casi d'uso**: un service riceve un comando, recupera i dati necessari tramite le port-in, delega la logica decisionale e di validazione alle entità del **domain** e infine persiste i cambiamenti tramite le port-out. I services non contengono logica di business, ma si occupano esclusivamente di coordinare il flusso operativo e gestire l'integrità delle transazioni (es. Listing 3).

Grazie a questa struttura, il sistema gode di una notevole resilienza; l'utilizzo delle interfacce assicura che il **disaccoppiamento** sia **totale**. Se in futuro fosse necessario migrare verso un diverso sistema di persistenza o un servizio cloud, sarebbe sufficiente **sviluppare un nuovo adapter** senza dover modificare il codice nel core applicativo.

Listing 1: Esempio Port-In (Contratto dei casi d'uso)

```
public interface BookingUseCase {
    void confirmBooking(Integer id);
    void cancelBooking(Integer id);

    //... altri metodi omessi ...
    List<Booking> getCustomerBookings(Integer customerId);
}
```

Listing 2: Esempio Port-Out (Requisiti verso il Database)

```
public interface BookingRepository {
    Integer save(Booking booking, TransactionContext context);
    void update(Booking booking, TransactionContext context);

    Optional<Booking> findById(Integer id, TransactionContext context);
    List<Integer> findOccupiedSpots(Integer beachId, LocalDate date,
        TransactionContext context);
}
```

Listing 3: Esempio Service (Orchestrazione e Transazionalità)

```

public class BookingService implements BookingUseCase {
    //... attributi e costruttore (dependency injection) ...

    @Override
    public void confirmBooking(Integer id) {
        transactionManager.executeInTransaction(context -> {
            Booking booking = getBookingOrThrow(id, context);

            //delegazione della logica di validazione al dominio
            booking.confirmBooking();

            //persistenza dello stato modificato
            bookingRepository.update(booking, context);
        });
    }
}

```

4.1.2 DDD-lite (Domain-Driven Design)

Per lo sviluppo del dominio è stato adottato l'approccio **DDD-lite**. Mentre l'architettura esagonale definisce i confini infrastrutturali, il **DDD** assicura che il nucleo del progetto rifletta fedelmente il linguaggio e le regole della realtà modellata.

Il cuore di questo approccio è il **Rich Domain Model**: a differenza di un modello anemico (fatto di soli getter e setter), nel nostro dominio **le entità** hanno il compito di **assicurarsi l'integrità dei dati** sia in fase di modifica che di istanziamento. Ad esempio: la classe **Beach** presenta metodi interni dedicati a controllare la presenza di tutti i dati obbligatori e il rispetto dei vincoli di business prima che l'oggetto possa essere effettivamente creato o modificato (es. Listing 4).

In questa architettura abbiamo distinto:

- **Entities**: oggetti con un'identità univoca che persiste nel tempo (es. Listing 4);
- **Value Objects**: oggetti definiti dai propri attributi e immutabili, implementati tramite i **Java Records** (es. Listing 5).

Altro elemento fondamentale dell'architettura sono gli **Aggregate**: unità logiche indivisibili che raggruppano entità strettamente correlate (come **Zone** e **Spot**) e i relativi **Value Objects**. In tale contesto, la **Beach** agisce come *Aggregate Root*, rappresentando l'unico punto di accesso responsabile di ogni variazione di stato all'interno del gruppo e garante dell'integrità dell'intero aggregato.

Il nostro approccio **DDD-lite** si distingue dal "Full DDD" per una precisa scelta di pragmatismo architetturale: laddove il DDD rigoroso impone spesso una gestione asincrona degli eventi per disaccoppiare i diversi aggregati, noi abbiamo optato per un'**orchestrazione sincrona**, così facendo la logica di coordinamento è affidata agli Application Services, i quali invocano sequenzialmente i metodi di dominio sulle Aggregate Root interessate. Questo modello permette di coordinare le azioni in maniera atomica all'interno di un'unica transazione **ACID** sul database relazionale, semplificando notevolmente le attività di test e manutenzione, pur preservando il totale incapsulamento della logica di business.

Listing 4: Esempio Entity (Integrità dello stato e logica di business)

```

public class Beach {
    private final Integer id; //identità univoca
    private BeachInventory beachInventory; //value object composito
    private boolean active;
    private boolean closed;

    //... costruttore omissso ...

    //metodo di dominio con validazione interna
    public void setActive(boolean active) {
        if (active) {
            validateActivationRequirements();
        }
        this.active = active;
    }

    private void validateActivationRequirements() {
        if (beachInventory == null)
            throw new IllegalStateException("Missing inventory");
        //... altri controlli di business ...
    }
}

```

Un aspetto centrale nella progettazione è la gestione del ciclo di vita dei **Value Objects**, improntata alla massima protezione dello stato tramite due principi cardine:

- **Ricostruzione del Modello:** le entità non vengono semplicemente lette dal database, ma *ricostituite*. Lo strato di persistenza fornisce i dati affinché il Core possa istanziare nuovamente l'entità, garantendo che i vincoli vengano verificati nel momento stesso della creazione in memoria.
- **Metodi Wither e Immutabilità:** invece di alterare lo stato di un'istanza esistente tramite *setter*, le modifiche generano una **nuova istanza** dell'oggetto (Listing 5). Questo garantisce l'**immutabilità** durante il ciclo di vita in memoria, facilitando il debugging e assicurando la consistenza tra gli strati applicativi.

Listing 5: Value Object: Integrità alla creazione e pattern Wither

```

public record BeachInventory(int countExtraSdraio, int countCamerini) {
    //costruttore compatto per validare le regole di business
    public BeachInventory {
        if (countExtraSdraio < 0 || countCamerini < 0) {
            throw new IllegalArgumentException("ERROR: counts cannot be negative");
        }
    }

    //metodo Wither: crea una nuova istanza immutabile con un dato aggiornato
    public BeachInventory withCountCamerini(int countCamerini) {
        return new BeachInventory(this.countExtraSdraio, countCamerini);
    }
}

```

4.1.3 Builder Pattern

Il pattern **Builder** è stato utilizzato per gestire l'istanziamento di classi caratterizzate da un elevato numero di parametri e attributi opzionali, evitando costruttori eccessivamente complessi e verbosi. Tramite una classe interna statica la **costruzione dell'oggetto** avviene in modo **guidato e leggibile** (es. Listing 6).

Il pattern è stato applicato a classi come **Address**, **Spot**, **Zone** e ai *value objects* associati a **Booking** e **Beach**. In particolare, nel caso della spiaggia, vista la densità di informazioni richieste, abbiamo suddiviso gli attributi in diversi *value objects*, ciascuno dei quali viene istanziato tramite il proprio builder. Questo permette di gestire con estrema facilità la configurazione di parametri non obbligatori, mantenendo il codice pulito e uniforme in tutto il progetto.

Listing 6: Implementazione del pattern Builder su un Value Object

```
public record BeachInventory(int countExtraSdraio, int countCamerini) {
    //attributi...

    //metodo Builder per costruire l'oggetto col pattern Builder
    public static Builder builder() {
        return new Builder();
    }

    //altri metodi...

    //pattern Builder per creazione BeachInventory facilitata
    public static class Builder {
        private int countExtraSdraio = 0;
        private int countExtraLettini = 0;
        private int countExtraSedie = 0;
        private int countCamerini = 0;

        public Builder countExtraSdraio(int val) {
            this.countExtraSdraio = val;
            return this; //permette il method chaining
        }

        //ogni attributo ha il suo metodo strutturato come sopra

        public BeachInventory build() {
            //la validazione avverrà automaticamente nel
            //costruttore del record
            return new BeachInventory(countExtraSdraio,
                                     countCamerini);
        }
    }
}
```

4.2 Struttura del codice

La struttura della directory del progetto è stata organizzata in base alla separazione dei package di Java e seguendo i principi dell'architettura software a livelli, descritta precedentemente.

```
s_balneare
├── .idea
├── .mvn
├── db-init
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.example.s_balneare
│   │   │       ├── application
│   │   │       │   ├── port
│   │   │       │   │   ├── in
│   │   │       │   │   └── out
│   │   │       │   └── service
│   │   │       ├── domain
│   │   │       ├── infrastructure.persistence.jdbc
│   │   │       ├── Main.java
│   │   │       └── module-info.java
│   │   └── resources
│   └── test
│       ├── java
│       │   └── com.example.s_balneare
│       │       ├── application
│       │       │   └── service
│       │       ├── domain
│       │       ├── AllTestsSuite.java
│       │       ├── ApplicationTestSuite.java
│       │       └── DomainTestSuite.java
├── target
├── .gitignore
└── docker-compose.yml
```

La directory dei test rispecchia la struttura principale per **mantenere gli Unit Test organizzati** (con package paralleli come `application` e `domain` e le relative test suite). L'adozione di queste pratiche garantisce un **progetto chiaro**, strutturato e **facilmente manutenibile**, evitando al contempo problemi legati alla visibilità dei package.

Ogni package interagisce con gli altri per svolgere compiti specifici, mantenendo una netta separazione tra il dominio applicativo (`domain`), i casi d'uso (`application`) e la gestione dei dati/interazioni con il database (`infrastructure.persistence.jdbc`). Tale approccio strutturato assicura che ogni componente del sistema abbia **una responsabilità ben definita**, rendendo il codice più facile da navigare, comprendere ed estendere. Inoltre facilita la collaborazione tra i membri del team, poiché ogni package può essere sviluppato e testato in modo indipendente.

4.3 Domain

Il livello di **Domain** rappresenta il nucleo fondante dell'applicazione, in cui risiedono le **regole di business** e la **modellazione dello stato**. In conformità ai principi del pattern **DDD-lite**, le classi non sono meri contenitori di dati, ma garantiscono autonomamente l'integrità del sistema. La struttura gerarchica e relazionale delle entità è sintetizzata nel diagramma UML riportato in Figura 7. Per il diagramma completo, con metodi e attributi di ciascuna classe, si veda il seguente PDF.

Dal punto di vista architettureale, la relazione tra **Entities** e **Value Objects** si concretizza tramite il principio della *composizione*: le entità **incapsulano** i value objects come propri attributi, **delegando** a questi ultimi la **gestione** di raggruppamenti logici e immutabili di **dati** (ad esempio, le informazioni generali o l'inventario di una spiaggia).

Per quanto riguarda la modellazione degli attori principali del sistema (*Customer*, *Owner* e *Admin*), si è optato per un approccio basato sull'**ereditarietà**. Tutti i ruoli estendono una classe base **User**, permettendo di **centralizzare le funzionalità** e le anagrafiche **comuni**, sfruttando il **polimorfismo** al fine di semplificare l'estensibilità del codice.

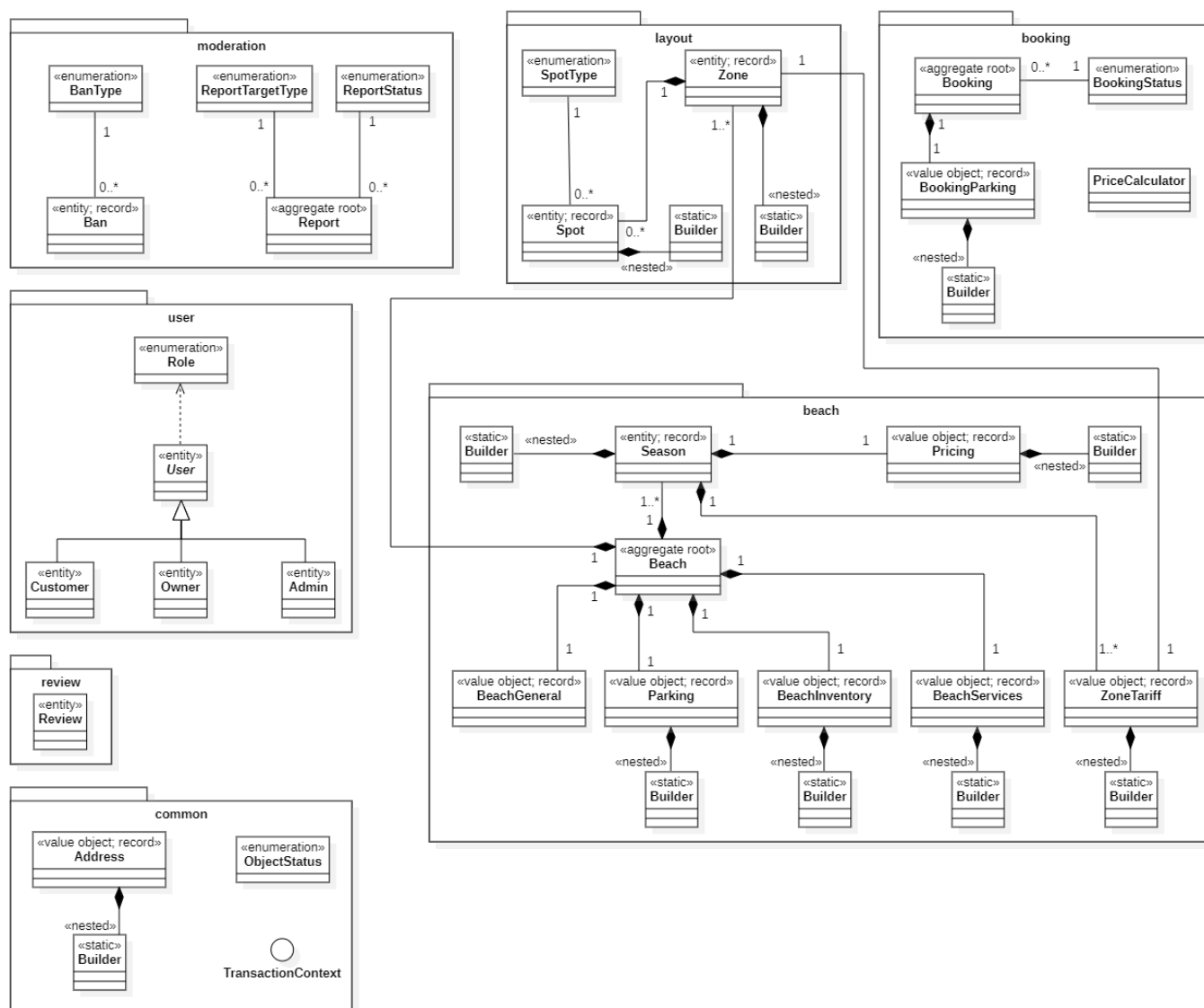


Figura 7: Domain UML

4.4 Application

Il livello applicativo ha il compito di interconnettere il cuore del progetto con il mondo esterno tramite gli strumenti precedentemente elencati.

4.4.1 Port-Out

Nel progetto, le **Port-Out** sono interfacce Java che **definiscono i requisiti di persistenza** e le **interazioni esterne** in modo astratto e universale. Questo approccio permette alla logica di business di invocare i metodi necessari senza dipendere dalla reale implementazione tecnologica sottostante. L'implementazione di queste interfacce è affidata al package **infrastructure.persistence.jdbc**, il quale ha il compito di tradurre le azioni dettate dal dominio in istruzioni JDBC per il database MySQL. Tale package include inoltre diversi **record** (evidenziati in rosso nella Figura 8), utilizzati come strutture dati (**DTO**) per veicolare le informazioni estratte dal database affinché siano adatte per essere elaborate dai metodi del core applicativo.



Figura 8: Struttura delle Port-Out

Il **TransactionManager** è l'interfaccia dedicata alla gestione del ciclo di vita delle transazioni, assicurando che il flusso di operazioni sul database rispetti le proprietà **ACID** (Atomicità, Consistenza, Isolamento e Durata). Il componente implementa il pattern **Unit of Work** che ha l'obiettivo di mantenere una lista di oggetti influenzati da una transazione e coordinare la scrittura delle modifiche.

Per l'implementazione di **TransactionManager** abbiamo scelto di adottare le *funzioni Lambda* in combinazione con l'utilizzo di un oggetto **TransactionContext**.

Quest'ultimo, è definito tramite un'interfaccia marker e assegnato all'interno dei metodi *executeInTransaction* della classe *jdbctransactionmanager* la quale implementa l'interfaccia **TransactionManager**. L'oggetto viene condiviso esplicitamente come parametro a ogni metodo delle repository coinvolto nel caso d'uso (es. `beachRepository.update(beach, context)`). Tale meccanismo garantisce che **tutte le operazioni** effettuate all'interno di un blocco transazionale **siano atomiche**: in caso di successo di ogni passaggio, il manager esegue il *commit* dei dati sul database; qualora venisse sollevata un'eccezione (es. un errore di sicurezza o validazione), il

manager interviene eseguendo un *rollback* automatico per preservare l'integrità e la consistenza dello stato persistente.

4.4.2 Port-In

Le **Port-In** (o *Driving Ports*) rappresentano i punti di ingresso all'esagono applicativo. Esse definiscono formalmente i casi d'uso e le azioni che gli attori esterni (come l'interfaccia utente o sistemi di test) possono richiedere al sistema.

In questa architettura, le port-in sono strutturate attraverso due componenti principali:

- **Use Case (Interfacce Java)**: definiscono i contratti delle operazioni di business. Esse dettano quali informazioni sono necessarie per eseguire un'azione e garantiscono che il mondo esterno interagisca con il core solo attraverso canali controllati;
- **Command e Request (Input Models)**: rappresentati spesso come *Java Record* (evidenziati in rosso nella Figura 9), questi oggetti fungono da contenitori di dati immutabili. Il loro compito è veicolare i parametri necessari per l'esecuzione del caso d'uso, permettendo una validazione sintattica preliminare prima che la richiesta raggiunga la logica di business.

L'implementazione concreta di queste interfacce è affidata al livello **service**. I servizi orchestrano il flusso chiamando correttamente i metodi definiti dalle port-out e delegando le regole decisionali alle entità del dominio, assicurando così che ogni richiesta venga elaborata in modo coerente e sicuro.

```

in/
├── beach/
├── booking/
├── common/
├── moderation/
├── review/
├── user/
booking/
├── BookingUseCase.java
├── CreateBookingCommand.java
├── CreateBookingUseCase.java
├── CreateManualBookingCommand.java
├── CreateManualBookingUseCase.java
moderation/
├── BanUseCase.java
├── CreateBanCommand.java
├── CreateReportCommand.java
├── CreateReportUseCase.java
├── ManageReportUseCase.java
beach/
├── BrowseBeachUseCase.java
├── CloseBeachUseCase.java
├── CreateBeachCommand.java
├── CreateBeachUseCase.java
├── ManageBeachUseCase.java
user/
├── CreateAdminRequest.java
├── CreateCustomerRequest.java
├── CreateOwnerRequest.java
├── CreateUserRequest.java
├── UserRegistrationUseCase.java
review/
├── CreateReviewCommand.java
├── ReviewUseCase.java
├── ViewBeachReviewsUseCase.java
common/
├── AddressUseCase.java

```

Figura 9: Struttura delle Port-In

5 Database

Per la gestione della persistenza dei dati è stato utilizzato il **DBMS MySQL**, configurato all'interno di un ambiente di containerizzazione **Docker**. L'interazione tra l'applicazione Java e il database avviene tramite l'interfaccia **JDBC**, utilizzando il driver ufficiale **MySQL Connector/J**.

L'integrità del dato è assicurata a livello nativo attraverso una **rigorosa definizione di vincoli** (**Foreign Key**, **Unique** e **Check**) implementati nello script di creazione. Tale approccio permette di delegare al database la validazione delle informazioni critiche, mentre le verifiche legate alla **logica di business** sono gestite programmaticamente all'interno del codice Java dell'applicazione.

5.1 Modello ER

Il modello Entity-Relationship (Figura 10) omette la relazione diretta tra le tabelle **beaches** e **bookings** per evitare ridondanze concettuali, essendo il legame già mediato dalle entità **zones** e **spots**. In fase di implementazione del database fisico, tuttavia, tale relazione è stata introdotta per ottimizzare le performance delle query di ricerca, riducendo la complessità delle operazioni di join.

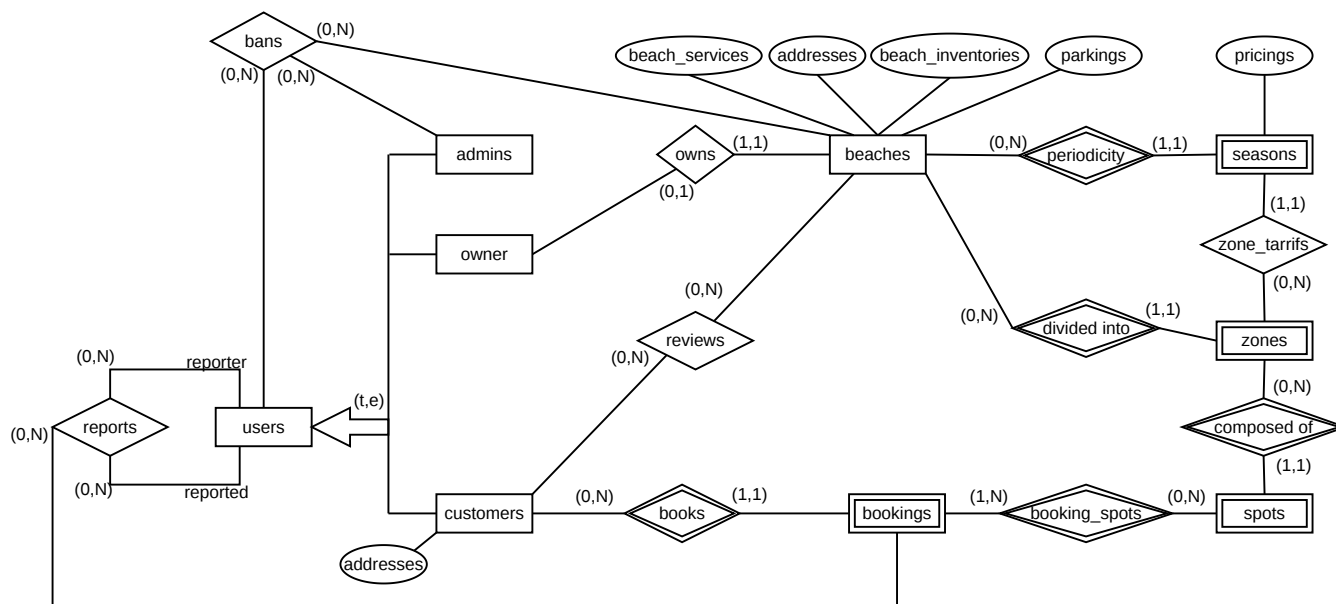


Figura 10: Modello Entity Relationship (Schema Concettuale)

5.2 Dizionario dei dati

Il dizionario (Figura 11 e Figura 12) presenta per **spots** e **bookings** attributi identificativi esterni derivanti dalle relazioni con altre entità. Per semplificare le interrogazioni e la realizzazione del database si è optato per l'utilizzo di una chiave primaria artificiale (**id**), preservando la valenza identificativa di questi attributi tramite vincoli **UNIQUE** e **NOT NULL**, come dettagliato nella tabella dei vincoli (Figura 13a).

Entità	Descrizione	Attributi	ID
Users	Generalizzazione degli utenti	name, surname, username, email, hashPassword	id
Admins	Amministratori di sistema	OTP	id (<i>Users</i>)
Owners	Gestori degli stabilimenti	active, OTP	id (<i>Users</i>)
Customers	Utenti (Clienti)	phoneNumber, active	id (<i>Users</i>)
Beaches	Stabilimenti balneari	name, description, phoneNumber, extraInfo, active, closed	id
Parkings	Attributo composto di Beaches. Info parcheggio	nAutoPark, nMotoPark, nElectricPark, CCTV	-
Beach Services	Attributo composto di Beaches. Servizi offerti	bathrooms, showers, pool, bar, restaurant, wifi	-
Beach Inventories	Attributo composto di Beaches. Inventario	countExtraSdraio, countExtraLettini, countExtraSedie, countCamerini	-
Addresses	Attributo composto. Indirizzi di utenti e spiagge	street, streetNumber, city, zipCode, country	-
Pricings	Listino prezzi base	priceLettino, priceSdraio, priceSedia, priceParking	-
Seasons	Periodi temporali (Stagionalità)	startDate, endDate, name	name, beachId (<i>Beaches</i>)
Zones	Aree geografiche della spiaggia	name	name, beachId (<i>Beaches</i>)
Spots	Postazioni prenotabili	type, row, column, zoneName (<i>Zones</i>), beachId (<i>Beaches</i>)	id
Bookings	Dati della prenotazione	date, extraSdraio, extraLettini, extraSedie, camerini, autoPark, motoPark, electricPark, totalPrice, status, callerName, callerPhone, customerId (<i>Customers</i>), beachId (<i>Beaches</i>)	id

Figura 11: Dizionario delle Entità

Relazioni	Descrizione	Entità coinvolte	Attributi
Bans	Gestione allontanamenti	Admins (0,N), Users (0,N), Beaches (0,N)	reason, banType, createdAt
Owns	Proprietà dello stabilimento	Owners (0,1), Beaches (1,1)	-
Reviews	Valutazioni degli utenti	Customers (0,N), Beaches (0,N)	rating, comment, createdAt
Books	Assegnazione prenotazione	Customers (0,N), Bookings (1,1)	-
Periodicity	Associazione stagionale	Beaches (0,N), Seasons (1,1)	-
Zone Tariffs	Prezzi per zona specifica	Seasons (1,1), Zones (0,N)	priceOmbrellone, priceTenda
Divided into	Partizione della spiaggia	Beaches (0,N), Zones (1,1)	-
Composed of	Layout dei posti	Zones (0,N), Spots (1,1)	-
Booking Spots	Occupazione fisica spot	Bookings (1,N), Spots (0,N)	date
Reports	Segnalazioni problemi	Users (0,N), Bookings (1,1)	status, description, createdAt

Figura 12: Dizionario delle Relazioni

5.3 Tabelle delle Regole

Di seguito vengono riportati i vincoli di integrità e le regole di derivazione che governano la logica della persistenza dei dati. In merito alla regola RD1 (Figura 13b), si è scelta la persistenza dell'attributo `totalPrice` nel database al fine di ridurre il carico computazionale di consultazione.

Codice	Regola di Vincolo (RV)
RV1	L'email e lo username di un utente (<i>Users</i>) devono essere univoci a livello di sistema.
RV2	Il numero di telefono (<i>phoneNumber</i>) di un cliente deve essere univoco.
RV3	Ogni spiaggia (<i>Beaches</i>) deve riferirsi obbligatoriamente a un proprietario (<i>Owners</i>) e a un indirizzo (<i>Addresses</i>) esistenti in modo univoco.
RV4	Il numero di posti auto, moto ed elettrici (<i>Parkings</i>) deve essere maggiore o uguale a zero.
RV5	Le quantità dell'inventario (<i>beach_inventories</i>), come ombrelloni e lettini, devono essere maggiori o uguali a zero.
RV6	Tutti i prezzi definiti nei listini (<i>Pricings</i>) e nelle tariffe per zona (<i>zone_tariffs</i>) devono essere maggiori o uguali a zero.
RV7	La data di inizio di una stagione (<i>startDate</i>) deve essere strettamente precedente alla data di fine (<i>endDate</i>).
RV8	Non possono esistere due postazioni (<i>Spots</i>) con la stessa riga e colonna nella medesima zona e spiaggia.
RV9	Le coordinate di riga (<i>row</i>) e colonna (<i>column</i>) di uno <i>Spot</i> devono essere maggiori o uguali a zero.
RV10	Un cliente non può effettuare più di una prenotazione (<i>Bookings</i>) per la stessa spiaggia nella medesima data.
RV11	Uno <i>Spot</i> non deve essere associato a più di una prenotazione (<i>booking_spots</i>) nella stessa data.
RV12	Le quantità di servizi extra richiesti in una prenotazione (lettini, sdraio, ecc.) devono essere maggiori o uguali a zero.
RV13	Un utente non può ricevere più di un ban attivo (<i>Bans</i>) per la stessa spiaggia o per l'intera applicazione.
RV14	Il rating di una recensione (<i>Reviews</i>) deve essere un valore intero compreso tra 1 e 5.
RV15	Un cliente può inserire al massimo una recensione per ogni spiaggia.
RV16	Ogni utente può emettere un solo report (<i>Reports</i>) per una specifica prenotazione.
RV17	Utilizzo identificatore surrogato (id) per Spots, deve esistere vincolo di unicità tra: <i>row</i> , <i>column</i> , <i>zoneName</i> e <i>beachId</i> .
RV18	Utilizzo identificatore surrogato (id) per Bookings, deve esistere vincolo di unicità tra: <i>customerId</i> , <i>beachId</i> e <i>date</i> .
RV19	La spiaggia associata a una prenotazione (<i>beachId</i> in <i>Bookings</i>) deve corrispondere alla spiaggia a cui appartengono i singoli posti prenotati (<i>beachId</i> in <i>Spots</i>).

(a) Tabella dei vincoli di integrità (RV)

Codice	Regola di Derivazione (RD)
RD1	Il prezzo totale (<i>totalPrice</i>) di una prenotazione si ottiene sommando la tariffa dello spot (basata su zona e stagione) al costo dei servizi extra e dei parcheggi prenotati (quantità moltiplicata per il rispettivo prezzo unitario).

(b) Tabella delle regole di derivazione (RD)

Figura 13: Tabelle delle Regole

6 Mockup

In questa sezione vengono presentati i mockup dell'interfaccia grafica, realizzati con lo scopo di **dimostrare visivamente** come i requisiti e le rigide regole di business (definite nel Domain-Driven Design) si traducano nell'esperienza utente finale.

Per ottimizzare i tempi di prototipazione e ottenere un design moderno e responsivo, sono stati impiegati due strumenti di generazione UI basati sull'Intelligenza Artificiale:

- **v0.dev (Vercel)**: utilizzato per generare i primi 4 mockup, focalizzati sull'esperienza Mobile (B2C – Business to Consumer) dedicata ai **Customer**.
- **bolt.new (StackBlitz)**: utilizzato per generare gli ultimi 4 mockup, focalizzati sulle Dashboard Desktop (B2B – Business to Business) destinate alla gestione avanzata da parte di **Owner** e **Admin**.

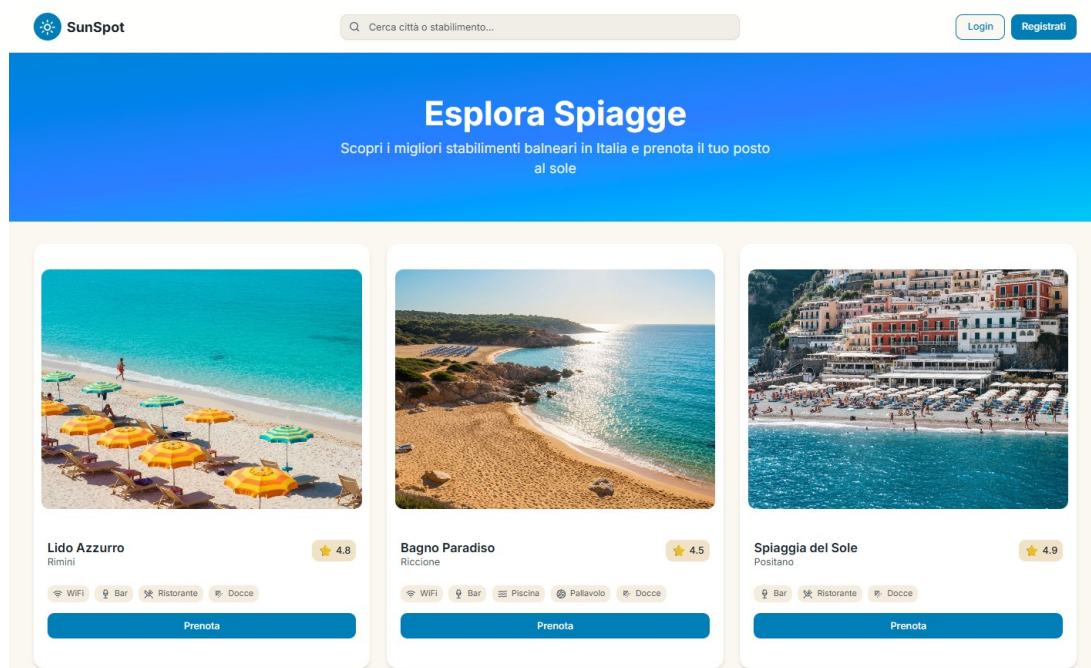


Figura 14: **Esplora Spiagge - Desktop (UC-04)**: interfaccia desktop per la ricerca nel catalogo. La vista interroga in sola lettura (CQRS tramite la `BeachCatalogQuery`) il database, mostrando esclusivamente le spiagge in stato *ATTIVO* ed esponendo la valutazione media.

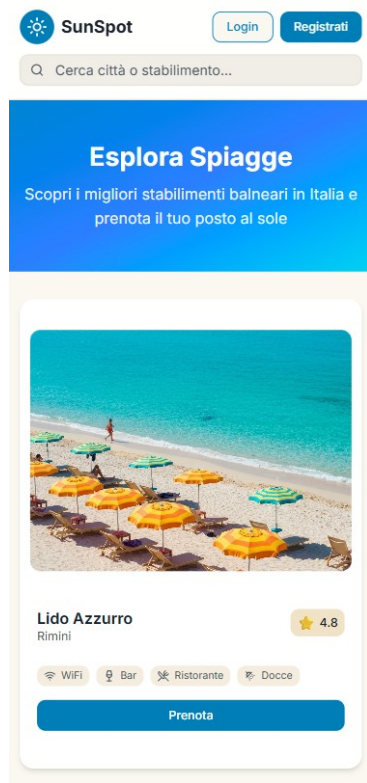


Figura 15: **Esplora Spiagge - Mobile (UC-04)**: versione responsiva (mobile) dell'interfaccia B2C per la consultazione del catalogo spiagge da parte di customers e guests.

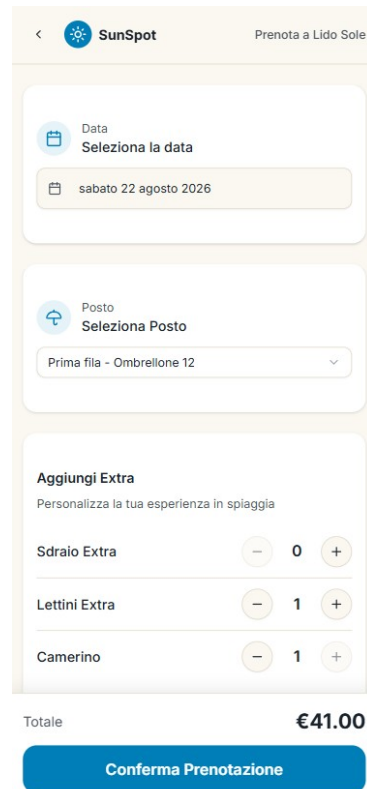


Figura 16: **Flusso di Prenotazione (UC-05)**: interfaccia mobile per la creazione di una prenotazione. Evidenzia l'applicazione delle regole di dominio: controlli stringenti sulla capienza massima di parcheggi ed extra, e il calcolo in tempo reale del prezzo totale (tramite `PriceCalculatorDomainService`).

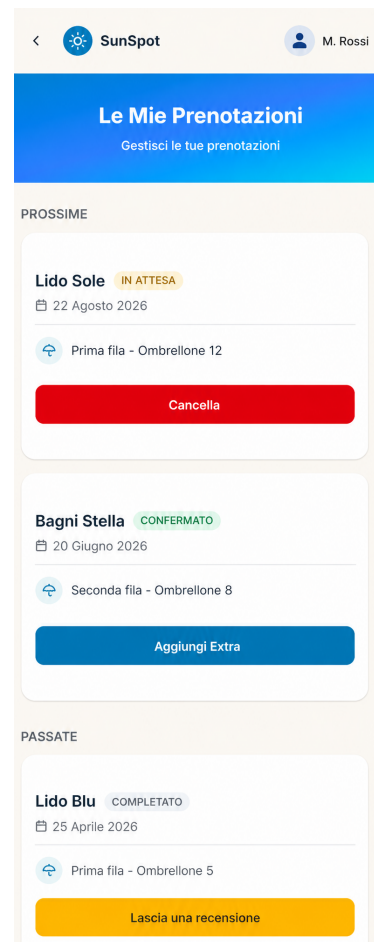


Figura 17: **Le Mie Prenotazioni e Recensioni (UC-06 & UC-07)**: dashboard mobile del customer. La UI riflette visivamente gli stati della prenotazione (*PENDING* e *CONFIRMED*). Viene inoltre mostrato il pulsante "Lascia una recensione", sbloccato dal backend esclusivamente per i soggiorni passati.

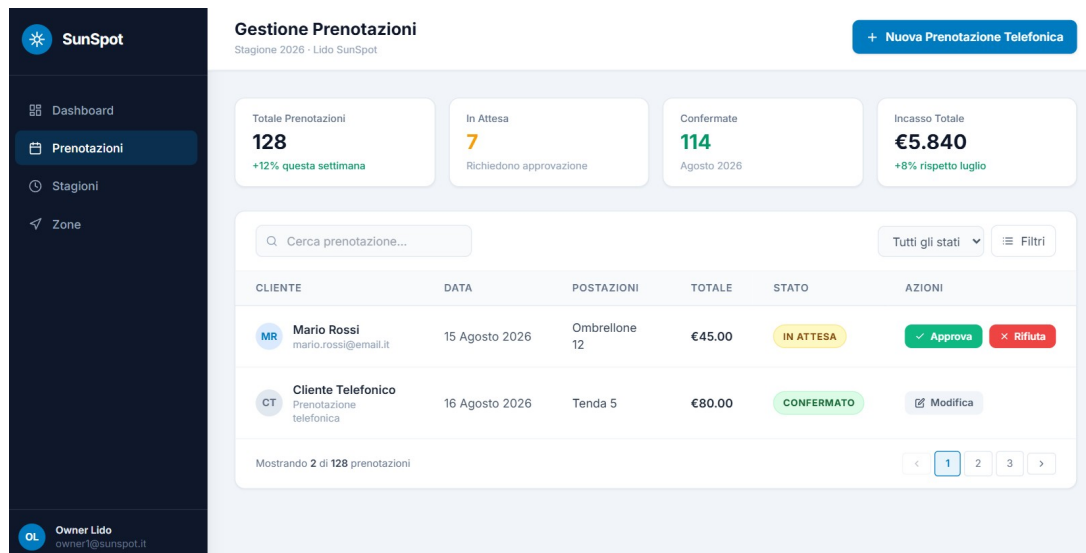


Figura 18: **Dashboard Prenotazioni Owner (UC-05-1a & UC-06)**: interfaccia desktop B2B. Mostra la lista delle richieste in attesa con i pulsanti per l'approvazione/rifiuto e il bottone per avviare la "Prenotazione Telefonica Manuale" (che supporta dati chiamante slegati dall'ID cliente).

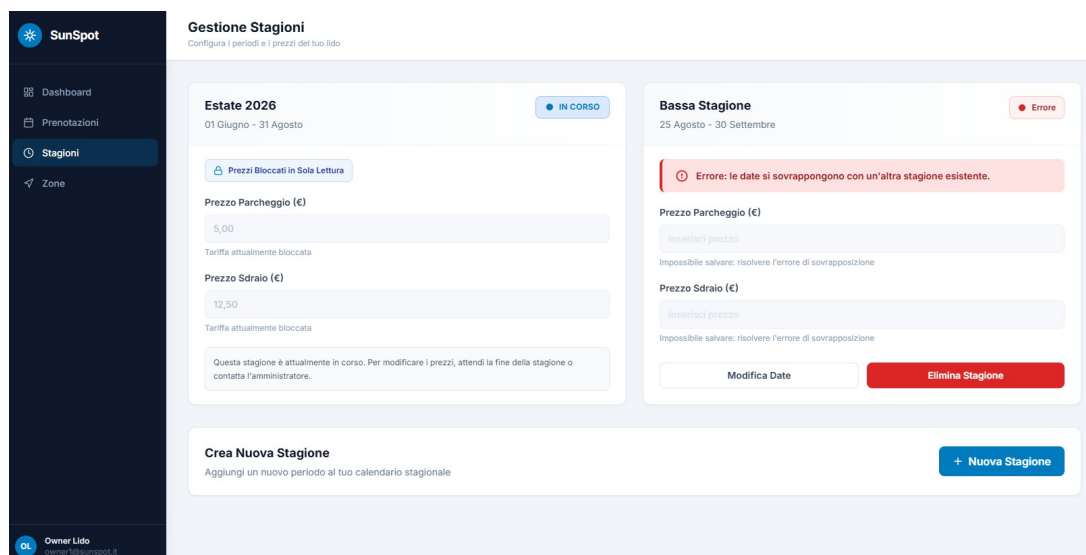


Figura 19: **Gestione Stagioni e Prezzi (UC-14)**: Il mockup dimostra l'imposizione delle regole di business del dominio: un avviso di errore per date sovrapposte (*overlap check*) e campi prezzo resi graficamente disabilitati (*sola lettura*) per garantire l'immutabilità finanziaria.

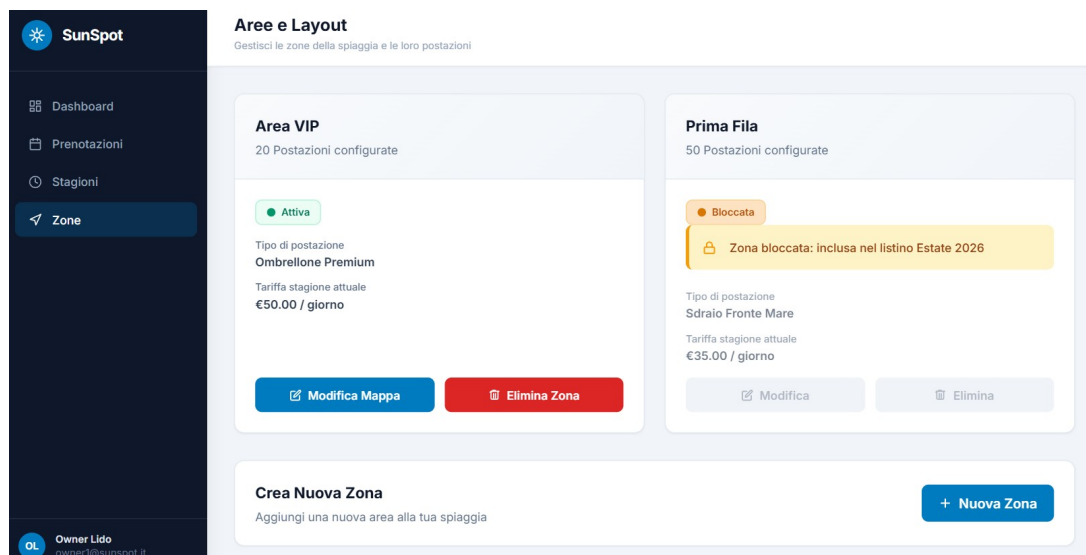


Figura 20: **Gestione Zone e Layout (UC-15)**: strumento di mappatura della spiaggia. La figura evidenzia il blocco logico applicato alle zone già associate a una stagione, impedendone la modifica strutturale a tutela dell'integrità dei dati.

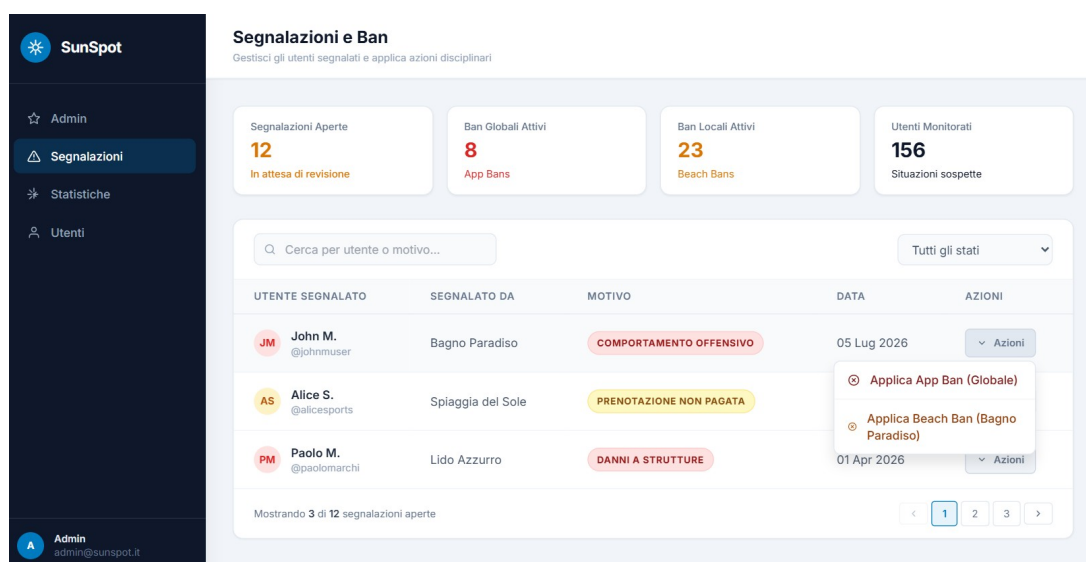


Figura 21: **Dashboard di Moderazione Admin (UC-18 & UC-19)**: interfaccia desktop riservata agli admins. Mostra la tabella delle segnalazioni evidenziando il comportamento polimorfo del ban implementato nel backend (scelta tra *app ban* globale e *beach ban* selettivo).

7 Unit Testing

7.1 Strategia e Obiettivi del Testing

La strategia di testing adottata per il sistema si basa sui principi della **Test-Driven e Behavior-Driven validation**, concentrandosi esclusivamente sui livelli *Domain* e *Application* dell'Architettura Esagonale. L'obiettivo principale è garantire la totale affidabilità delle regole di business e dell'orchestrazione, mantenendo i test rapidi e isolati dall'infrastruttura reale (Database).

Nello specifico, il testing è stato diviso in due macro-aree:

- **Domain Layer (Test Puri):** i test sulle entità di dominio (es. *Beach*, *Booking*) e sui Value Object verificano il rispetto dei vincoli architetturali (invarianti). Garantiscono che gli oggetti non possano mai essere istanziati in stati inconsistenti (es. coordinate negative, date sovrapposte) e validano la corretta implementazione della macchina a stati (es. transizioni da *PENDING* a *CONFIRMED*) senza alcuna dipendenza esterna.
- **Application Layer (Test con Mock):** i test sui Service verificano la corretta orchestrazione dei casi d'uso. Utilizzando la libreria **Mockito**, le porte di uscita e il **TransactionManager** sono stati simulati (*mockati*). Questo ha permesso di testare analiticamente i percorsi di successo (*Happy Paths*) e le barriere di sicurezza (*Sad Paths*), come il blocco di operazioni non autorizzate, i conflitti di capienza (inventario/parcheggi) e le logiche di rollback, garantendo il pieno rispetto dei requisiti utente.

7.2 Copertura dei Test: Classi e Metodi

Domain Layer

I test di questo layer verificano la pura logica di business, l'integrità dei dati e i vincoli architetturali, senza alcuna dipendenza da framework o database esterni.

- **Aggregato Beach:**
 - **Integrità e Macchina a Stati:** validazione dei requisiti minimi per l'attivazione e blocco rigoroso delle modifiche strutturali qualora la spiaggia sia operativa (*ACTIVE*) o permanentemente chiusa (*CLOSED*);
 - **Logica Temporale (stagioni):** verifica dell'algoritmo di anti-sovrapposizione delle date (*overlap check*) e dell'impossibilità di accorciare stagioni già definite;
 - **Logica Spaziale (zone):** test sul blocco di immutabilità che impedisce di eliminare, rinominare o alterare zone già vincolate a una stagione esistente.
- **Aggregato Booking e PriceCalculator:**
 - **Macchina a Stati e Sicurezza:** transizioni di stato sicure (*PENDING* → *CONFIRMED* → *CANCELLED/REJECTED*) e regola di declassamento automatico: ogni modifica ad una prenotazione già confermata forza il ritorno allo stato *PENDING*;
 - **Integrità dei Dati:** verifica della differenziazione tra prenotazioni di utenti registrati e chiamanti manuali (pulizia automatica del numero di telefono);
 - **Domain Service:** test rigorosi sul **PriceCalculator** per garantire che il calcolo del totale incroci correttamente la data, la zona dello spot e il tariffario stagionale, bloccando richieste anomale (es. spot inesistenti o date fuori stagione).

- **Gerarchia User (Customer, Owner, Admin):**
 - **Validazione Ereditata:** test astratti che garantiscono la validazione di email (formato e lunghezza), username e anagrafica per tutte le implementazioni;
 - **Comportamenti Specifici:** gestione corretta dei flag di sicurezza (disattivazione OTP per Owner/Admin) e logica di chiusura permanente dell'account per il Customer.
- **Moderazione (Ban, Report, Review):**
 - **Invarianti di Dominio:** impossibilità di auto-segnalarsi, limiti stringenti sulle valutazioni (1-5 stelle) e controlli temporali (divieto di recensioni o report nel futuro);
 - **Coerenza Polimorfica:** verifica delle dipendenze corrette in base al tipo di ban (un ban globale non deve avere ID spiaggia, un ban locale lo richiede obbligatoriamente).
- **Value Objects e Layout (Inventory, Services, Pricing, Spot, ecc.):**
 - **Contratti e Immutabilità:** difesa assoluta contro valori matematicamente illogici (quantità o prezzi negativi);
 - **Pattern Creazionali:** verifica dell'inizializzazione sicura tramite *Static Factories* (`empty()`, `none()`) e pattern *Builder* (incluso il costruttore copia);
 - **Copie Difensive:** verifica che liste interne (come l'elenco di spot in una zona) non possano essere alterate dall'esterno una volta istanziate.

Application Layer (Services)

I test di questo layer verificano la corretta orchestrazione dei flussi applicativi, l'integrazione tra aggregati distinti e l'applicazione delle regole di sicurezza, simulando le dipendenze esterne (Database e API) tramite l'uso di **Mockito**.

- **Gestione Spiaggia (ManageBeachService, CreateBeachService, CloseBeachService):**
 - **Creazione e Vincoli:** verifica del blocco architettonico della relazione 1:1 tra Owner e Beach e salvataggio automatico in stato *draft* (inattivo) in caso di configurazione parziale (UC-10);
 - **Gestione Operativa:** test sui *Delta Updates* per l'aggiornamento di inventario e parcheggi, garantendo che riduzioni di capienza non invalidino prenotazioni future già confermate (UC-11);
 - **Ciclo di Vita:** verifica del processo di attivazione/disattivazione (UC-12) e chiusura irreversibile (UC-13), che garantisce l'annullamento a cascata delle prenotazioni future tramite il `BookingRepository`.
- **Motore di Prenotazione (CreateBookingService, BookingService, CreateManualBookingService):**
 - **Orchestrazione Sicura:** verifica incrociata (**Cross-Aggregate Validation**) prima del salvataggio: controllo disponibilità residua di parcheggi e inventario, assenza di ban attivi per l'utente, e verifica che gli spot selezionati appartengano effettivamente alla spiaggia richiesta;
 - **Prenotazioni Manuali (Owner):** Test del flusso B2B (UC-05 1a) che salta il check dell'utente (*Guest calling*) e impone l'approvazione automatica (stato *CONFIRMED*);
 - **Vincoli Temporali:** blocco categorico di aggiornamenti, cancellazioni o approvazioni su prenotazioni la cui data coincida con quella odierna o passata.

- **Autenticazione e Utenze (`AuthenticationService`, `CreateUserService`, `ManageUserService`):**
 - **Gateway di Sicurezza:** test completi sul processo di Login (UC-01), che bloccano tentativi di accesso con credenziali errate, account disattivati o ban attivi (sia a livello applicativo che di struttura);
 - **Transazioni Atomiche:** verifica del pattern *Template Method* per la creazione utenti: hashing sicuro della password e rollback automatico in caso di violazione dei vincoli (es. email/username duplicati);
 - **Gestione Sicura Dati:** test sulle operazioni sensibili (cambio email o chiusura account Customer) che richiedono la ri-validazione della password attuale per essere autorizzate.
- **Moderazione e Recensioni (`BanService`, `CreateReportService`, `ReviewService`):**
 - **Polimorfismo Sanzionatorio:** test della logica di `Ban` che dirama azioni differenti: un *app ban* elimina tutte le prenotazioni future (per i customer) o chiude l'intera struttura (per gli owner), mentre un *beach ban* agisce solo a livello locale;
 - **Precondizioni Rigorose:** un utente può lasciare una recensione (UC-07) o un report solo se il sistema certifica (tramite query su `BookingRepository`) un soggiorno effettivamente completato nel passato e l'assenza di provvedimenti disciplinari attivi;
 - **Prevenzione IDOR:** verifica di sicurezza (Insecure Direct Object Reference) che impedisce a un utente di eliminare una recensione o un report non di sua proprietà.
- **Query Services (`BrowseBeachService`, `ManageReportService`, `ViewBeachReviewsService`):**
 - **Read Models:** test focalizzati sull'interrogazione ottimizzata dei dati, verificando la corretta aggregazione dei DTO (es. `BeachSummary`) filtrati per keyword, paginazione o stato attivo, e l'unione corretta di liste multiple (es. report ricevuti e inviati).

7.3 Esecuzione dei Test tramite JUnit 5 Suites

Per facilitare l'esecuzione e l'integrazione continua (CI/CD), l'infrastruttura di test è stata organizzata utilizzando la funzionalità **Test Suite** fornita da **JUnit 5** (modulo `junit-platform-suite`). Sono state create **tre classi orchestratrici** principali:

1. **DomainTestSuite:** Questa suite include esclusivamente i pacchetti sotto `com.example.s_balneare.domain`. È progettata per essere eseguita in pochi millisecondi. Essendo i test di dominio completamente isolati, questa suite viene utilizzata frequentemente durante lo sviluppo attivo per garantire che le modifiche non infrangano le regole matematiche o i vincoli logici degli aggregati.
2. **ApplicationTestSuite:** Questa suite isola i test del pacchetto `com.example.s_balneare.application`. Richiede leggermente più memoria a causa dell'inizializzazione del framework Mockito per la simulazione delle porte di uscita, ma garantisce che tutta la logica di orchestrazione, validazione incrociata e gestione delle transazioni funzioni correttamente.
3. **AllTestsSuite:** È la suite globale che esegue in sequenza sia i test di Dominio che quelli Applicativi.